2010376103 CSE4261 Assignment-15

OMOR NILOY

October 2025

1 Introduction

Natural Language Processing (NLP) has witnessed significant improvements with the introduction of transformer-based models. Among them, BERT (Bidirectional Encoder Representations from Transformers) stands out as one of the most influential architectures. BERT is pretrained on large corpora using unsupervised objectives, allowing it to learn deep bidirectional representations. The pretrained model can then be fine-tuned for various downstream tasks such as sentiment classification, question answering, and named entity recognition.

In this assignment, the primary objective is to build a BERT model from scratch, train it using the Masked Language Modeling (MLM) task, and then fine-tune it for sentiment classification on the IMDB movie reviews dataset. This exercise demonstrates the two-step paradigm of pretraining and fine-tuning in modern NLP pipelines, highlighting the transferability of learned representations.

2 Dataset and Preprocessing

The IMDB dataset consists of 50,000 movie reviews, evenly split between positive and negative sentiments. It is a standard benchmark dataset for binary sentiment classification. Data preprocessing is crucial for NLP tasks to ensure that the model learns meaningful patterns.

2.1 Steps in Preprocessing

- **Data Loading:** All review texts are read from the `aclImdb` dataset folders for both training and testing splits.
- **Text Standardization:** Each review is converted to lowercase and punctuation is removed to reduce vocabulary sparsity. Special attention is given to keep a `[MASK]` token for the MLM task.
- **Tokenization and Vectorization:** Reviews are converted into integer sequences using Keras' `TextVectorization` layer. The vocabulary size is set to 30,000 with a maximum sequence length of 256 tokens. This step ensures consistent input sizes for BERT.
- **Masked Language Modeling Preparation:** To train the model on MLM, 15% of tokens are randomly masked. 90% of these masked tokens are replaced with the `[MASK]` token, 10% remain unchanged, and another 10% are replaced with a random token from the vocabulary. Labels for unmasked tokens are ignored to prevent unnecessary loss computation.

3 BERT Model Architecture

BERT leverages the Transformer architecture, which uses self-attention to capture contextual relationships between tokens. Unlike traditional models, BERT is bidirectional, meaning it considers both left and right contexts simultaneously.

3.1 Model Components

- **Embedding Layer:** The model starts with word embeddings and positional embeddings to capture token semantics and sequence order.
- **Transformer Encoder Layers:** Each encoder layer consists of:
  - Multi-Head Self-Attention: Captures dependencies between all tokens in a sequence.
  - Feed-Forward Neural Network: Applies nonlinear transformations to the attention outputs.
  - Layer Normalization and Dropout: Stabilizes training and prevents overfitting.
- **MLM Head:** A dense layer with softmax activation predicts the masked tokens in the input sequence.

3.2 Hyperparameters

- Maximum sequence length: 256
- Vocabulary size: 30,000
- Embedding dimension: 128
- Number of attention heads: 8
- Feed-forward dimension: 128
- Number of encoder layers: 1
- Batch size: 32
- Learning rate: 0.001

4 Masked Language Modeling

Masked Language Modeling (MLM) is an unsupervised pretraining objective where certain tokens are masked and the model predicts these tokens. This encourages the model to learn contextual representations.

4.1 Training Procedure

- The preprocessed dataset is converted into masked input sequences along with corresponding target labels.
- The model is trained using `SparseCategoricalCrossentropy` loss.
- Training is conducted for 5 epochs with Adam optimizer.
- A custom callback evaluates sample predictions at the end of each epoch to monitor the model's learning progress.

Listing 1: Training BERT MLM model

```
bert_masked_model = create_masked_language_bert_model()
bert_masked_model.fit(mlm_ds, epochs=5, callbacks=[generator_callback])
bert_masked_model.save("bert_mlm_imdb.keras")
```

5 Fine-Tuning for Sentiment Classification

After pretraining, the BERT encoder is used as a feature extractor. A classifier is added on top for binary sentiment classification.

5.1 Classifier Architecture

- Global Max Pooling over token embeddings.
- Dense hidden layer with 64 units and ReLU activation.
- Output layer with sigmoid activation for binary prediction.

5.2 Training Strategy

1. Initially, BERT encoder layers are frozen to train only the classifier.
2. Once the classifier achieves stable performance, the encoder layers are unfrozen and the entire model is fine-tuned.
3. The model is trained for 5 epochs in both phases using the Adam optimizer.

Listing 2: Fine-tuning BERT for Sentiment Classification

```
history = classifer_model.fit(
    train_classifier_ds,
    epochs=5,
    validation_data=test_classifier_ds,
)
```

6 Results and Discussion

6.1 Masked Language Modeling

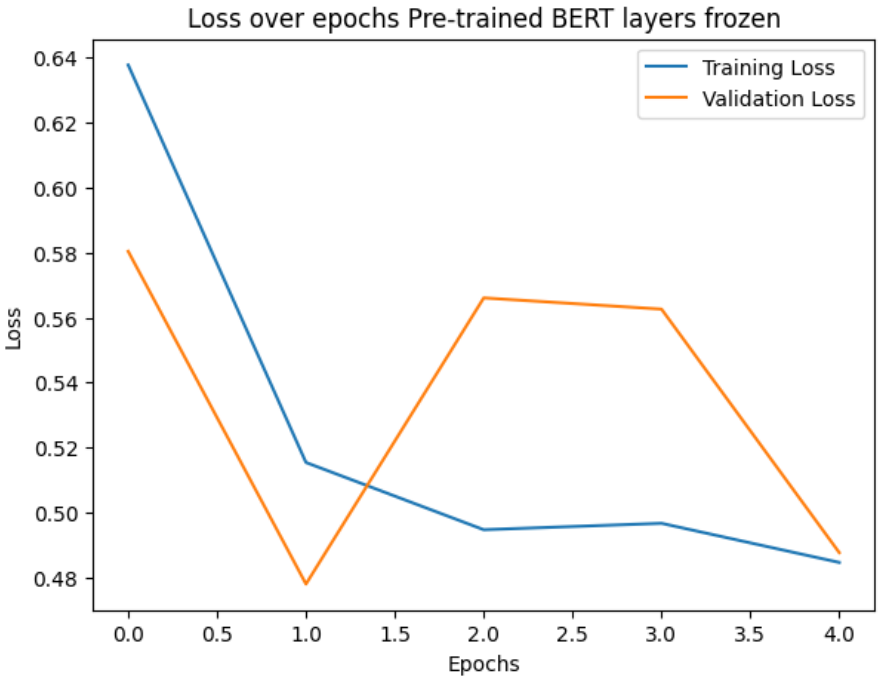

Figure 1: Masked Language Modeling: Training loss and sample masked token predictions.

6.2 Sentiment Classification

6.2.1 Frozen BERT Layers

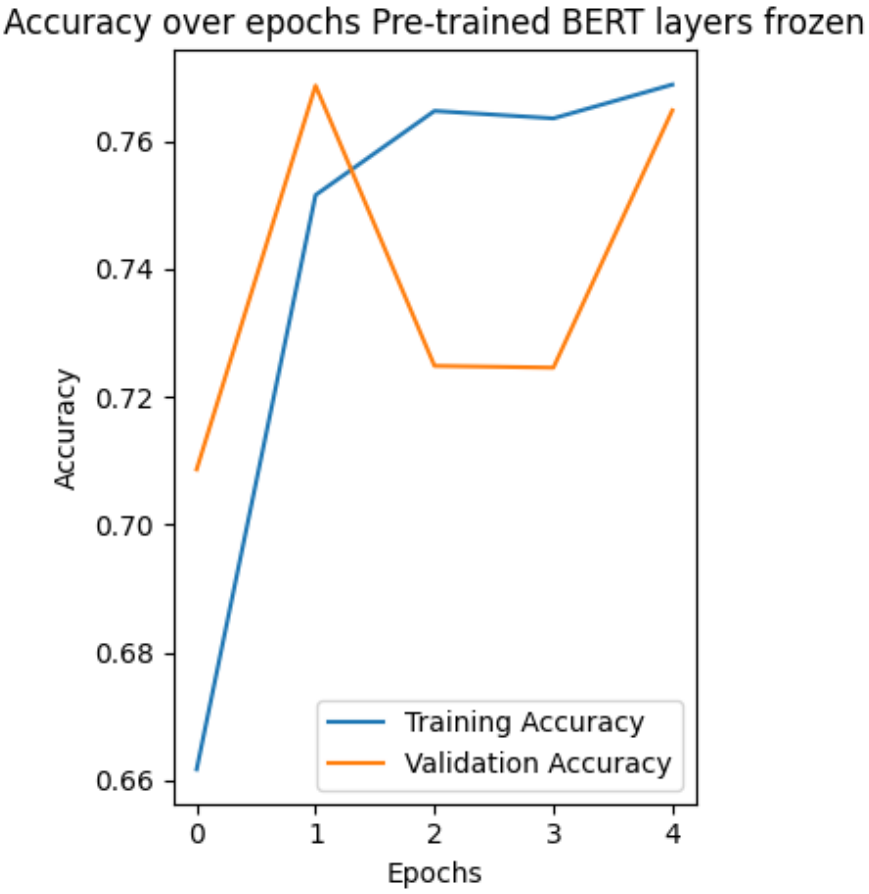

Figure 2: Classifier training with frozen BERT encoder. Loss and Accuracy over epochs.

6.2.2 Unfrozen BERT Layers (Fine-tuning)

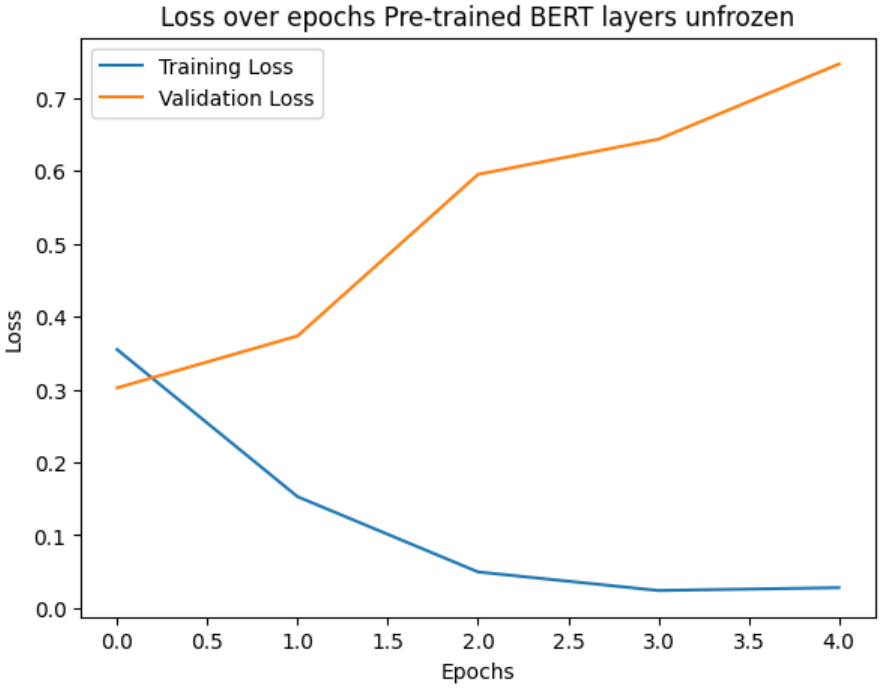

Figure 3: Classifier fine-tuning with BERT encoder unfrozen. Loss and Accuracy over epochs.

7 Conclusion

In this assignment, we successfully implemented a BERT model from scratch and trained it using the Masked Language Modeling task. We then fine-tuned the pretrained encoder for sentiment classification on the IMDB dataset. The experiments demonstrate that pretraining with MLM allows BERT to learn rich contextual representations that significantly improve performance on downstream tasks. This exercise highlights the effectiveness of transfer learning in modern NLP pipelines and provides practical experience in building and fine-tuning transformer-based models.

8 Code

Github code link